

G-quAI: Predicting Gastrointestinal Cancer Using Quantum AI

Phase 3 Report

Ana Beatriz Morgado, Nuno Batista, Oscar Ferraz, Sagar Silva Pratapsi,
Jorge Lobo, Gabriel Falcão

April 10, 2026

Abstract

Please add a short summary of what has been achieved in Phase 3 of the OQI Use Case and what were the most important findings. Max 250 words.

Relevant SDGs

- **SDG 3: Good Health and Well-Being** - Gastrointestinal (GI) cancers, particularly colorectal cancer (CRC), represent a significant global health challenge, with over 2 million new cases diagnosed annually [?, ?]. Our work focuses on increasing the precision of early detection via Quantum AI to directly impact survival rates and reduce the burden of non-communicable diseases.
- **SDG 9: Industry, Innovation and Infrastructure** - Current scientific advancements enable the use of non-invasive, high-quality imaging and miniaturized electronic devices that can travel through the GI tract [?]. We build upon this infrastructure by integrating quantum-enhanced algorithms into the diagnostic pipeline, representing a major leap in medical AI infrastructure.
- **SDG 10: Reduced Inequalities** - Classical deep learning often demands massive, expensive computational clusters. By optimizing Quantum Neural Networks for low-resolution imaging, we aim to develop diagnostic tools that are energy-efficient and deployable in low-resource settings or remote regions.
- **SDG 13: Climate Action** - We are exploring the potential of Quantum Neural Networks to offer a more sustainable alternative to the high-energy consumption of training large-scale classical AI models.

- **SDG 17: Partnerships for the Goals** - This project is a multi-disciplinary effort, bridging the gap between quantum computing research, medical imaging experts, and international health organizations.

Link to GitHub Repository

The development process and all simulation codes are maintained in a central repository to ensure modularity and reproducibility. The repository includes the newly developed CLI-based dataset generator, the FRQI encoding scripts, and the variational circuit training pipeline optimized for GPU backends.

<https://github.com/G-quAI>

1 Setting up the Simulation

The simulation phase of the G-quAI project was designed to bridge the gap between theoretical quantum circuit design and real-world medical application.

As mentioned in our previous reports, we are pursuing two distinct quantum computing paradigms:

- **Quantum Neural Networks (QNNs):** Focused on gate-based variational circuits, optimized for superconducting architectures.
- **ZZ-AQCNN (Analog Quantum Convolutional Neural Networks):** Focused on neutral-atom processors, utilizing physics-grounded analog Hamiltonian evolution and higher-order entanglement correlations.

1.1 Quantum Neural Networks (QNNs)

The following subsections detail the simulation environment, data preparation, and architectural choices for the Quantum Neural Network (QNN) implementation.

1.1.1 Simulation Environment and Classical Hardware

To handle the exponential computational demands of simulating high-resolution quantum states (up to 128×128 pixels), we utilized a multi-tier hardware strategy. Simulations were conducted on high-performance servers at the Instituto de Telecomunicações (IT) at the University of Coimbra.

- **Primary Compute Nodes:**
 - **rtx3095:** Equipped with NVIDIA RTX high-memory GPUs, utilized for smaller image resolutions simulations.
 - **cray-4:** Used for the most intensive 128×128 simulations and long-duration grid searches.
- **Software Stack:**

- Virtual Environments: Managed via Anaconda, with dedicated environments for `pennylane_cpu` and `pennylane_jax`.
- Backends: We transitioned from the standard `default.qubit` to the C++ based `lightning.qubit` and subsequently to `lightning.gpu` for CUDA-accelerated state-vector simulation.
- Optimization Framework: We integrated JAX for Just-In-Time (JIT) compilation, which allowed for significant speedups in training time.

1.1.2 Data Acquisition and Pre-processing Pipeline

A critical achievement in Phase 3 was the development of a fully automated, CLI-driven data pipeline. This allowed for rapid iteration across different image resolutions and dataset characteristics. Given the hardware constraints of current quantum simulators, we utilized synthetic data to systematically evaluate the QNN's ability to recognize patterns under controlled conditions before moving to real-world medical images.

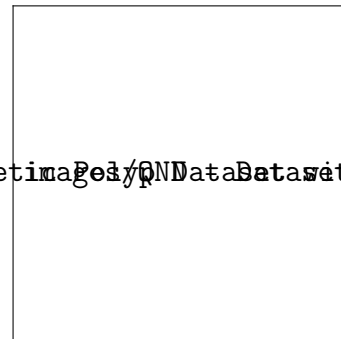
Dataset Sources and Categories

To rigorously test the QNN's classification limits, we categorized our data into three distinct types:

1. **Noiseless Synthetic Polyp Dataset**: These images represent the idealized "clean" case. They consist of a textured background (simulating the mucosa) and a clearly defined geometric shape representing the polyp. This dataset establishes the baseline for the model's ability to learn spatial features using the FRQI encoding.



(a) Positive: Noiseless Polyp



(b) Negative: Healthy Mucosa

2. **Noisy Synthetic Polyp Dataset**: Building upon the clean dataset, we introduced controlled Gaussian noise. This simulates the grainy output often found in low-light endoscopic video feeds. Testing on this dataset is crucial for evaluating the robustness of the QNN, as quantum states are notoriously sensitive to input perturbations.

images/oqi_logo.png

images/oqi_text.png

images/QNN - Dataset

Images/Synthetic

images/Polyp/QNN Dataset

Images/Synthetic

Images/Synthetic

(a) Positive: Noisy Polyp

(b) Negative: Noisy Healthy Mucosa

3. **Simple Black and White Dataset:** A binary-mask style dataset used to verify the mathematical correctness of the mapping and the basic convergence of the loss function without the complexity of background textures.

images/QNN - Dataset

Images/Synthetic

images/QNN - Dataset

Images/Synthetic

Images/Synthetic

(a) Positive: Black and White Polyp

(b) Negative: Black and White Healthy Mucosa

Resolution Hierarchy and Native Data Generation

A key feature of our Phase 3 development is the ability to generate datasets natively at varying resolutions. Rather than downsampling from a high-resolution "master" image, our script utilizes the `-image_size` CLI argument to construct the synthetic scenes directly at the target scale.

- **Native Resolution Generation:** By generating images directly at 4×4 , 8×8 , 16×16 , 32×32 , and 64×64 , we ensure that the geometric features (the polyp) and the mucosal textures are rendered according to the specific qubit capacity of the corresponding quantum circuit. This allows for a "fair" assessment of the FRQI encoding without artifacts introduced by classical downsampling algorithms.
- **Visual Ground Truth:** The 128×128 samples (as seen in the previous figures) serve as our visual "ground truth." They allow us to verify that the generation logic correctly simulates the medical context (polyp vs. healthy tissue) before we scale the logic down to the resolutions manageable by current quantum simulators.

1.1.3 Quantum Algorithm Mapping and Architecture

The core of our diagnostic engine is a hybrid Quantum-Classical Neural Network. We focused on maximizing the information density of the quantum state while maintaining a circuit depth compatible with the Noise-Intermediate Scale Quantum (NISQ) era.

Image Encoding: The FRQI Mathematical Framework

We implemented the Flexible Representation of Quantum Images (FRQI). This method encodes an image of size $2^n \times 2^n$ by mapping spatial coordinates to the computational basis and pixel intensities to the probability amplitudes of a single color qubit. The quantum state representing the image is given by:

$$|\psi\rangle = \frac{1}{N} \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} |i\rangle|j\rangle(\cos(\theta_{ij})|0\rangle + \sin(\theta_{ij})|1\rangle) \quad (1)$$

Where:

- $N = 2^n$ represents the dimension of the square image.
- $|i\rangle$ and $|j\rangle$ are the quantum states representing the vertical and horizontal pixel positions, respectively.
- $\cos(\theta_{ij})|0\rangle + \sin(\theta_{ij})|1\rangle$ represents the color (intensity) information encoded in the ancilla qubit.
- $\theta_{ij} \in [0, \pi/2]$ is the angle derived from the normalized grayscale intensity of the pixel at position (i, j) .

A significant advantage of FRQI is its logarithmic scaling of spatial information. To represent an $N \times N$ image, we require n qubits for the row index, n qubits for the column index, and 1 ancilla qubit for the color, totaling $2n + 1$ qubits. In this phase, we simulated various resolutions to observe the trade-off between image detail and simulation complexity:

- 4×4 **images** ($n = 2$): Required 5 qubits.
- 16×16 **images** ($n = 4$): Required 9 qubits.
- 64×64 **images** ($n = 6$): Required 13 qubits.

While the theoretical preparation of the $|\psi\rangle$ state involves a sequence of Hadamard gates followed by Controlled-Rotation gates, our simulation pipeline utilizes a high-performance statevector preparation method. We compute the normalized amplitudes classically and load them into the simulator using the `qml.StatePrep` function. This allows us to focus the quantum "learning" computational budget on the variational layers rather than the overhead of state preparation, which is essential for the deep-circuit grid searches (up to 22 layers) conducted in this phase.

Variational Circuit Design: HEA vs. Parametric Entanglement

We explored two distinct architectural approaches for the Variational Quantum Circuit (VQC):

- **Basic Hardware-Efficient Ansatz (HEA):** Utilizing `qml.BasicEntanglerLayers`, this architecture consists of RY rotations followed by a chain of CNOT gates.
- **Parametric Entanglement (CRZ-based):** In our more advanced scripts, we moved beyond static CNOT gates to Controlled-RZ (CRZ) gates. This allows the model to "learn" the optimal level of entanglement between spatial coordinates and color information, effectively making the entanglement itself a trainable parameter.

The Hybrid Model: Trainable Observable Layer

To bridge the quantum output with classical classification, we implemented a `TrainableObsLayer`. Rather than taking a simple expectation value, we treat the expectation values of all n qubits as a vector:

$$\hat{y} = \text{sign} \left(\sum_{i=1}^n w_i \langle \psi(\theta, \phi) | Z_i | \psi(\theta, \phi) \rangle \right) \quad (2)$$

The weights w_i are optimized classically, allowing the model to prioritize specific spatial frequencies or features learned during the quantum evolution.

1.1.4 Training Strategies and Pipeline Evolution

A major technical milestone in Phase 3 was the transition from standard deep learning loops to high-performance quantum-classical pipelines.

From PyTorch to JAX: High-Performance Optimization

Initially, we utilized PyTorch (`qml.qnn.TorchLayer`) for its flexibility. However, to scale to 64×64 and 128×128 resolutions, we developed a "Stateless" pipeline using JAX and Optax. The technical advantages included:

- **Just-In-Time (JIT) Compilation:** Compiling the entire training step into XLA-optimized machine code, significantly reducing the overhead of the Python interpreter.
- **Vectorized Mapping (vmap):** JAX allowed us to eliminate manual Python loops over batches. By vectorizing the model forward pass, we achieved massive parallelization on the cray-4 GPU clusters.
- **Stateless Pure Functions:** Unlike PyTorch, our JAX models own no internal state, making them more stable for long-duration grid searches and complex gradient computations.

Automated Hyper-parameter Optimization (Grid Search)

To find the "Golden Configuration", we implemented a multi-dimensional Grid Search. We systematically evaluated 20 different depths (from 1 to 20 layers) in combination with various learning rates, batch sizes and optimizers. This automated process generated over 300 unique experiment directories, each containing loss histories, ROC curves, and Confusion Matrices, providing the statistical backbone for the results presented in Section 2.

Progressive Layer Sequencing and Weight Transfer

To ensure robust model convergence and the continuous improvement of diagnostic metrics,

we developed a Progressive Training Sequence. This strategy serves a dual purpose: it mitigates the "Barren Plateau" phenomenon (where gradients vanish as circuit depth increases) and establishes a framework for incremental learning by reusing the optimization results from shallower configurations.

1. The process begins with a single-layer circuit.
2. The model is trained for a predefined "Learning Budget" (e.g., 5,000 or 10,000 epochs, set via the `-epochs-per-layer` argument). This ensures that each complexity level has an equal opportunity to adapt to the data before further parameters are added.
3. After the epoch budget is exhausted, a new variational layer is appended to the circuit. Crucially, the weights from the previous N layers are transferred and utilized as the "Warm Start" for the next phase.
4. The new $(N + 1)$ -th layer is initialized with weights set to zero. This effectively makes the new layer an identity operation at the start of its training cycle, ensuring that the model begins the next phase with exactly the same performance it had at the end of the previous phase.

This strategy ensures that the model always builds upon its previous successes. By reusing the learned weights, we provide the classical optimizer with a high-quality starting position in the Hilbert space, allowing the metrics to improve monotonically as the expressive power of the circuit grows. This method proved vital for successfully training circuits up to a depth of 20 layers, providing a level of stability that random global initialization could not achieve.

1.2 ZZ-AQCNN (Analog Quantum Convolutional Neural Networks)

This approach builds upon our previous methodology: **QGARS** (Quantum Guided Autoencoder with Reservoir Surrogate). While the QGARS approach demonstrated the potential of Rydberg-state encoding for polyp detection using a task-guided latent space, the current ZZ-AQCNN framework represents a strategic pivot toward a more hardware-aware architecture.

1.2.1 Previous Work: QGARS Performance

In our prior work evaluating the QGARS framework, we compared various dimensionality reduction techniques coupled with Quantum Reservoir Computing (QRC) on medical and standard datasets. As shown in Table ??, the QGARS method significantly outperformed unguided Autoencoders (AE + QRC) and Principal Component Analysis (PCA + QRC) on complex medical datasets such as the Synthetic Polyp Dataset and CVC-ClinicDB.

The transition from a global reservoir with a surrogate-driven autoencoder to a patch-based quantum kernel is motivated by several key technical advantages:

1. **Spatial Locality:** By processing 3x3 image patches directly, we preserve the spatial correlations of the medical scans, mirroring the inductive bias of classical CNNs.

Table 1: Mean $\pm$ standard deviation of the classification accuracy (%) for QRC with different dimensionality reduction methods (from previous work).

Method	Synthetic	CVC-ClinicDB	MNIST(0/1)
PCA + QRC	79.20 ± 2.53	84.95 ± 2.31	97.70 ± 0.97
AE + QRC	77.50 ± 2.70	73.75 ± 1.90	87.05 ± 2.09
QGARS + QRC	89.45 ± 2.06	88.90 ± 1.65	97.00 ± 2.37

2. **Native 2D Topology:** ZZ-AQCNN leverages the natural 2D arrangement of neutral-atom arrays. Unlike the 1D chains often utilized in reservoir models, our diverse topologies (Kings, Ring, Grid) exploit the Rydberg blockade effect across multiple spatial dimensions.
3. **Efficiency and Scalability:** The kernel approach eliminates the need for the complex surrogate-driven training of auto-encoders required by QGARS, focusing computational resources on the high-dimensional feature mapping provided by the analog Hamiltonian.

The simulation phase for the ZZ-AQCNN architecture focuses on patch-based quantum kernels rather than a global reservoir.

1.2.2 Dataset and Data Pre-processing

To validate our ZZ-AQCNN architecture, we utilized the BreastMNIST dataset from the MedMNIST v2 collection. This dataset consists of low-resolution (28x28) ultrasound images categorized into benign and malignant cases. It serves as an excellent benchmark for evaluating the effectiveness of our patch-based feature extraction method on highly noisy, variance-heavy medical scans.

1.2.3 Software Stack and Simulation Environment

The simulation and optimization pipeline for the ZZ-AQCNN was built upon a robust open-source quantum machine learning stack:

- **Quantum Simulation:** We utilized **PennyLane** (`default.qubit` and `lightning.qubit`) for both digital gate simulation and analog Hamiltonian evolution.
- **High-Performance Computing:** The training loops were accelerated using **JAX** and **Optax** for Just-In-Time (JIT) compilation and vectorized execution.
- **Hyperparameter Tuning:** We employed **Optuna** to orchestrate massive 1,000-trial grid searches for the optimal classical head and quantum topology configurations.
- **Experiment Tracking:** All trials, validation seeds, and learning curves were actively tracked and aggregated using **Weights & Biases (W&B)**.

1.2.4 Encoding Schemes and Hamiltonian Evolution

We distinguish between two fundamentally different ways of mapping classical image data $\mathbf{x}$ into the quantum Hilbert space.

Digital Encoding: In the digital configuration, the data from the pixels is all encoded in the initial states of the atoms at $t = 0$. Each pixel $x_i \in [0, \pi]$ is used as the parameter for a rotation gate $RY(x_i)$, followed by a Hadamard gate H . This prepares a data-dependent initial state $|\psi(\mathbf{x})\rangle$, which then evolves under a static Hamiltonian H_{fixed} for a duration T .

$$|\Psi_{\text{out}}\rangle = e^{-iH_{\text{fixed}}T} \left(\bigotimes_{i=1}^N H \cdot RY(x_i) \right) |0\rangle^{\otimes N} \quad (3)$$

Analog Encoding: In our prioritized analog mode, we eliminate initial gate operations ($|\psi_{t=0}\rangle = |0\rangle^{\otimes N}$) and instead inject the data directly into the system’s dynamics. Each pixel x_i is mapped to the local detuning coefficient δ_i of the Rydberg Hamiltonian. The quantum state evolves according to the image data throughout the entire evolution period $[0, T]$, allowing for more complex, non-linear feature extraction.

$$|\Psi_{\text{out}}\rangle = \mathcal{T} \exp \left(-i \int_0^T H(\mathbf{x}, t) dt \right) |0\rangle^{\otimes N} \quad (4)$$

where $H(\mathbf{x}, t) = \frac{\Omega(t)}{2} \sum_i \sigma_X^{(i)} - \Delta(t) \sum_i n_i + \sum_{i<j} J_{ij} n_i n_j + \sum_i x_i n_i$. This scheme is highly hardware-native as it leverages the natural control fields of neutral-atom processors.

1.2.5 ZZ Entanglement Correlators

To capture richer features, we measure both $\langle \sigma_Z^{(i)} \rangle$ and $\langle \sigma_Z^{(i)} \sigma_Z^{(j)} \rangle$ for all $i < j$. For a 9-qubit kernel, this increases the feature count from 9 to 45 ($9 + \binom{9}{2}$), potentially providing the classical head with deeper insights into the quantum state’s entanglement structure.

1.2.6 Topological Diversity and Feature Extraction

The strength of the ZZ-AQCNN approach lies in the structural diversity of its kernels. By manipulating the physical coordinates of the 9 qubits, we effectively create different quantum receptive fields. We use 8 topologies: Kings, Horizontal, Vertical, Cross, Ring, Chain, Star, and Grid.

2 Results

2.1 Quantum Neural Networks (QNNs)

The results presented in this section document the systematic evaluation of the G-quAI Quantum Neural Network (QNN) during Phase 3. Our simulations were designed to validate the theoretical models proposed in Phase 2 by testing the performance of the FRQI-based classifier across varying image resolutions, circuit depths, and noise conditions. All primary experimental trials in this section were conducted using the Noisy Synthetic Polyp Dataset to ensure the model’s robustness against realistic endoscopic artifacts.

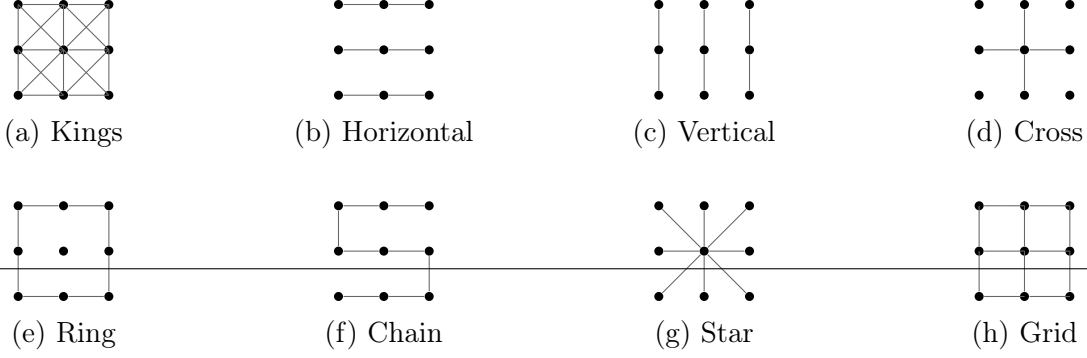


Figure 1: Visualization of the 8 kernel topologies. Nodes represent qubits; lines represent strong Hamiltonian interactions (J_{ij}). Isolated nodes represent qubits displaced to the *FAR* position.

2.1.1 Preliminary CPU-based Trials

The initial stage of Phase 3 focused on establishing a performance baseline using our CPU-based PennyLane and PyTorch integration. These trials utilized the `lightning.qubit` device for state-vector simulation, providing a benchmark for Accuracy, Precision, Recall, F1-Score, and AUC.

To understand the scaling limits of the QNN, we tested four native image resolutions: 4×4 , 8×8 , 16×16 , and 32×32 pixels. These sizes were chosen to fit within the memory constraints of our institute’s CPU-based infrastructure while maintaining the logarithmic qubit scaling of the FRQI encoding ($n = 2 \times \log_2(N) + 1$).

For this baseline phase, we utilized the Noisy Synthetic Polyp Dataset consisting of 250 synthetic images with 500 training epochs per run. To determine the effect of circuit complexity on diagnostic sensitivity, we tested four distinct depths of the Hardware-Efficient Ansatz: 3, 5, 7, and 15 layers.

As illustrated in Figure ??, the 16×16 resolution emerged as a representative case for analysis.

Initial results from the CPU trials (the 16×16 case and others summarized in the Figures ?? to ?? of the Appendices) reveal critical trends in how the QNN handles low-resolution medical features in the presence of noise:

- **Resolution Impact:** At the lowest resolution (4×4), the model struggled to achieve high AUC, as the extreme downscaling removed the geometric features necessary to distinguish between healthy tissue and lesions. However, as resolution increased to 16×16 and 32×32 , the QNN showed a marked improvement in feature recognition, despite the increased simulation time.
- **Depth-to-Accuracy Correlation:** Across all resolutions, we observed that increasing the circuit depth from 3 to 15 layers significantly stabilized the training loss. At 15 layers, the model displayed its highest Accuracy and Precision scores.
- **Baseline Metrics:** In the 16×16 noisy trials, the QNN demonstrated consistent convergence. While noise slightly degraded absolute accuracy compared to noiseless

images/oqi_logo.png

images/oqi_text.png

images/CPU/16x16.png

images/footer.jpg

Figure 2: Metrics results for 16×16 resolution (CPU Trials, 500 epochs).

images/oqi_logo.png

images/oqi_text.png

simulations, the Weighted Observable Layer allowed the model to maintain an AUC above the 0.7 baseline, proving that the hybrid architecture can effectively "filter" non-essential spatial information even in sub-optimal conditions.

As shown in Figure ??, regardless of the specific image resolution, the average processing time per epoch consistently exceeded the 60-second threshold. In the context of our experimental design—which requires thousands of epochs to achieve convergence and multiple trials for hyperparameter tuning—this latency became a critical bottleneck. For a standard 500-epoch run, a single trial would require over 8 hours of continuous CPU computation. This made large-scale grid searches and high-resolution simulations practically unfeasible within a reasonable research timeframe.

images/CPU/time.png

images/footer.jpg

Figure 3: Average Training Time per Epoch vs. Image Resolution (CPU Trials).

Faced with these constraints, it became necessary to explore high-performance alternatives to accelerate the quantum simulation and optimize the training throughput.

2.1.2 High-Performance JAX-based Simulations

To overcome the computational bottleneck identified in the preliminary trials, we re-engineered the diagnostic pipeline to utilize a high-performance stack based on JAX. This transition represented a fundamental shift from the "eager execution" of PyTorch to an optimized,

compiled execution model.

The implementation introduced several key optimizations that transformed simulation throughput on the `rtx3095` and `cray-4` nodes:

- **Just-In-Time (JIT) Compilation:** By utilizing JAX's `@jit` decorator, the entire training step—including the FRQI state preparation, the variational circuit execution, and the gradient update—is compiled into XLA-optimized machine code. This fuses operations and eliminates the Python interpreter overhead that hindered the CPU trials.
- **Vectorized Batch Processing (`vmap`):** In our CPU implementation, batches were processed through manual loops. The JAX-based pipeline utilizes the `vmap` (vectorized map) transformation, allowing the simulator to process entire batches in parallel on the GPU.
- **Stateless Execution:** By implementing the model as a "pure function", we ensured optimal memory management, which proved essential for scaling to 64×64 resolutions (13 qubits).

As a direct result of this migration, the epoch time for a 16×16 resolution dropped from over 60 seconds to less than 0.5 seconds, as can be seen in ??, representing a speedup of over $120\times$. This efficiency enabled us to double the training duration to 1,000 epochs per run and perform deeper searches across larger image sizes.

As illustrated in Figure ??, the 16×16 resolution again serves as our primary representative case for analysis under this high-performance framework.

The results from the JAX-based campaign on the Noisy Synthetic Polyp Dataset (summarized for other resolutions in Figures ?? to ?? of the Appendices) demonstrate a significant stabilization of diagnostic performance.

This $120\times$ speedup fundamentally transformed our research methodology. By reducing the computational cost of a single trial from hours to minutes, we moved beyond individual tests and were able to conduct a systematic, large-scale exploration of the model's hyper-parameter space. This transition from manual configuration to automated optimization is detailed in the following subsection regarding our Grid Search campaign.

2.1.3 Hyper-parameter Optimization: Automated Grid Search

With the high-performance JAX engine established, we conducted an exhaustive Grid Search on the Noisy Synthetic Polyp Dataset to identify the "Golden Configuration". That is, the specific combination of circuit depth, learning rate, and batch size that yields peak diagnostic performance. By automating this process, we were able to map the performance landscape across 20 different depths (1 to 20 layers) and multiple optimization parameters, generating the statistical evidence required to validate our QNN architecture.

The Grid Search was executed across three primary dimensions:

- **Circuit Depth:** We evaluated the impact of expressivity by testing every depth from 1 to 20 layers using the Hardware-Efficient Ansatz.

images/oqi_logo.png

images/oqi_text.png

images/JAX/time.png

images/footer.jpg

Figure 4: Average Training Time per Epoch vs. Image Resolution (JAX Trials).

images/oqi_logo.png

images/oqi_text.png

images/JAX/16x16.png

images/footer.jpg

Figure 5: Metrics results for 16×16 resolution (JAX Trials, 1,000 epochs).

- **Learning Rates:** We tested four distinct rates (0.1, 0.05, 0.01, 0.001) to identify the threshold for stable convergence in the quantum Hilbert space.
- **Batch Sizes:** We compared sizes of 16, 32, and 64 to determine the optimal gradient estimation accuracy vs. computational throughput.

Despite the added overhead of managing multiple parameter combinations, the automated Grid Search maintained a high level of throughput. As shown in Figure ??, the average processing time per epoch remained remarkably stable. While the management of the grid search parameters caused a slight increase in latency compared to the single-run JAX tests, the training time per epoch remained consistently below 1 second for all resolutions up to 128×128 . This sub-second performance was the key enabler for our Phase 3 development, allowing us to evaluate hundreds of configurations in a timeframe that would have taken weeks on a standard CPU backend.

images/Grid Search/time.png

images/footer.jpg

Figure 6: Average Training Time per Epoch vs. Image Resolution (Grid Search Campaign).

As illustrated in Figure ??, the results for the 16×16 resolution demonstrate the critical importance of circuit depth in medical pattern recognition. Other results from this Grid Search analysis are summarized in Figures ?? to ?? in the Appendices.

After an initial analysis of the multi-dimensional grid search results, it was clear that we did not need to continue testing for so many combinations. Consequently, to optimize our

images/oqi_logo.png

images/oqi_text.png

images/Grid Search/16x16.png

images/footer.jpg

Figure 7: Grid Search results for 16×16 resolution.

computational resources, we chose to fix the Learning Rate at 0.05 and the Batch Size at 32 for the subsequent, more intensive phases of the project.

2.1.4 Convergence and Performance under Fixed Parameters

By focusing on the identified optimal learning rate (0.05) and batch size (32), we conducted a series of high-duration simulations with a budget of 5,000 epochs per configuration. This extended training period was critical to observe the asymptotic behavior of the QNN and to ensure that the performance metrics reached a true steady-state, effectively filtering out transient noise in the optimization landscape.

As illustrated in Figure ??, the results for the 16×16 resolution on the Noisy Synthetic Polyp Dataset show a refined performance profile compared to the initial 1,000-epoch sweeps.

images/Grid Search Fixed/16x16.png

images/footer.jpg

Figure 8: Fixed-parameter Grid Search results for 16×16 resolution.

Extended results for all other resolutions under these fixed parameters are summarized in Figures ?? to ?? in the Appendices.

The analysis of these long-duration trials reveals several nuanced technical insights regarding the model's behavior:

- **Anomalous Depth Performance (16×16):** For the 16×16 resolution, we observed that the model achieved remarkably high performance at specific circuit depths, with the **AUC reaching peaks over 0.90**. However, this performance was not always monotonic. While some depths produced excellent diagnostic results, others nearby showed unexpected dips. At this stage, the exact correlation between specific layer counts and mucosal feature extraction remains unclear; it appears that certain circuit architectures "align" more effectively with the FRQI-encoded spatial features than others, a phenomenon that warrants further theoretical investigation into quantum feature mapping.
- **The Scaling Bottleneck ($> 32 \times 32$):** A critical finding emerged when scaling beyond the 32×32 resolution. As shown in the 64×64 and 128×128 results (see Appendix ??), the model's performance significantly degrades, with all metrics flatlining near 0.5.

2.1.5 Baseline Verification: Simple Black and White Dataset

To further investigate the performance degradation observed at higher resolutions, we conducted a final control experiment using the Simple Black and White Dataset. The objective was to eliminate the complexity of mucosal background textures and simulated noise, providing an idealized "best-case scenario" for the QNN. This allowed us to verify the mathematical correctness of the FRQI mapping and determine if the scaling bottleneck was a result of pattern complexity or a structural limitation of the current variational architecture.

As illustrated in Figure ??, the results for the 16×16 resolution confirm the effectiveness of the model when spatial features are clearly defined and noise-free. Other results from this



Figure 9: Black and White Baseline results for 16×16 resolution.

baseline analysis are summarized in Figures ?? to ?? in the Appendices.

The findings from this simplified baseline are crucial for our architectural evaluation:

- **Mathematical Validation and Depth Anomalies:** For the 4×4 , 8×8 , and 16×16 resolutions, the QNN demonstrated near-perfect metrics, with Accuracy, Recall, and Precision all approaching 1.0. This confirms that the FRQI mapping and the basic variational logic are mathematically sound. However, we observed isolated "performance dips" at specific circuit depths where the metrics unexpectedly degraded. Once again, at this stage, we still do not have a definitive theoretical explanation for why certain specific depths fail while those immediately surrounding them succeed; this suggests a non-linear sensitivity in the variational landscape that requires further study.
- **The Precision-Recall Mismatch at Scale:** As we moved to larger resolutions (32×32 and above), a distinct failure pattern emerged. While the Precision almost always remained at 1.0 (indicating zero false positives), the Recall consistently dropped to approximately 0.5. As a result, the overall Accuracy fell to roughly 0.8.

This specific statistical profile suggests that at higher resolutions, the model becomes "overly conservative." It correctly identifies all healthy samples (high True Negatives) but only manages to identify exactly half of the polyp samples. This "partial recognition" indicates that the current circuit is likely only "seeing" a subset of the spatial features in the 13 to 15-qubit Hilbert space. Even without the distraction of mucosal textures, the structural bottleneck of the Hardware-Efficient Ansatz prevents it from achieving the full diagnostic sensitivity needed for larger image scales.

2.1.6 Optimized Training: Progressive Layer Sequencing

To address the performance degradation and the "Barren Plateau" phenomenon observed in the static grid search trials, we implemented a Progressive Layer Sequencing strategy. This approach shifts the paradigm from training a fixed-depth circuit to an incremental growth model. By building upon previously learned features, we aimed to provide the QNN with a stable optimization path toward higher-resolution classification.

The progressive training lifecycle for each image resolution followed a rigorous protocol:

- **Layer-wise Evolution:** The circuit starts as a single variational layer. Once the 10,000-epoch budget is completed, a new layer is appended.
- **Weight Transfer:** The weights from the previous N layers are "transferred" to the new, deeper circuit.
- **Identity Initialization:** The $(N + 1)$ -th layer is initialized with weights set to zero. Since we utilized Controlled-RZ (CRZ) gates for entanglement instead of static CNOTs, this initialization makes the new layer an identity operation, ensuring that the model starts each new phase with the exact performance of the previous one.

As illustrated in Figure ??, the results for the 16×16 **Noiseless Synthetic Polyp Dataset** represent the peak performance achieved during Phase 3.

Other results from this progressive sequence analysis are summarized in Figures ?? to ?? in the Appendices.

The technical impact of this strategy is summarized below:

images/oqi_logo.png

images/oqi_text.png

images/Progressive Sequence/16x16.png

images/footer.jpg

Figure 10: Progressive Sequence results for 16×16 resolution.

- **Monotonic Metric Improvement:** Unlike the static grid search where results were often erratic, the progressive sequence demonstrates a stable, monotonically increasing trend in all metrics.
- **Breakthrough at High Resolutions:** The most significant achievement of this method was its performance on larger image sizes. While the static grid search resulted in "random guessing" for resolutions above 32×32 , the Progressive Sequence enabled the model to maintain a learning trend even at 128×128 (see Appendix ??). This confirms that the bottleneck was not the image size itself, but the difficulty of optimizing a high-qubit Hilbert space from a random starting point.

The effectiveness of the progressive approach is best visualized through the training loss evolution. As shown in Figures ?? and ??, we monitored the "Continuous Loss" across the entire training lifecycle and the "Minimum Loss" achieved at each depth.

While the layer-wise evolution described above successfully achieved a monotonic improvement in diagnostic metrics, a detailed analysis of the Continuous Loss (Figure ?? and other figures in Appendix ??) reveals significant "spikes" or "jumps" at each transition point. Despite the use of identity initialization for new weights, these discontinuities suggest that the gradient information and optimizer momentum were being partially lost whenever the circuit structure was redefined. Effectively, the optimizer was being forced to restart its search on a new, higher-dimensional surface without the benefit of its previous moving averages and history.

To mitigate this and ensure a truly seamless learning curve, we evolved the pipeline into a Progressive Sequence with a Single-Circuit Architecture.

2.1.7 The Single-Circuit Dynamic Approach

The final technical evolution in Phase 3 involved moving from a growing circuit to a static, high-capacity architecture with dynamic execution. As observed in the previous subsection, redefining the circuit structure at each layer transition caused significant optimization "shocks", as the optimizer's internal state (momentum and gradient history) was reset. To resolve this, we implemented the Progressive Sequence with a Single-Circuit Architecture. The methodology involves initializing a full 20-layer circuit at the start. Rather than adding new hardware components, we utilize a static `active_depth` parameter to "unmask" layers as the training progresses. This ensures that the optimizer maintains its historical context across all 200,000 epochs (10,000 per layer), providing a seamless and highly stable learning trajectory.

The primary success of this approach is visible in the global training history. As shown in Figure ??, the "Full Continuous Loss" for the 16×16 resolution eliminates the massive spikes seen in the previous method.

From a performance standpoint, managing a static 20-layer circuit introduces a slight computational overhead compared to the simpler JAX trials. As shown in Figure ??, the training time per epoch approximately doubled. However, this remains orders of magnitude more efficient than our initial CPU-based trials. While the CPU required over 60 seconds per epoch for a 16×16 image, this refined JAX approach completes the same task in roughly

images/oqi_logo.png

images/oqi_text.png

images/Progressive Sequence/16x16_continuousLoss.png

images/footer.jpg

Figure 11: Continuous Training Loss for 16×16 resolution.

images/oqi_logo.png

images/oqi_text.png

images/Progressive Sequence/16x16_minLoss.png

Figure 12: Minimum Training Loss for 16×16 resolution.

1 second, maintaining the high-throughput necessary for the 200,000-epoch total training budget.

As illustrated in Figure ??, the results for the 16×16 **Noiseless Synthetic Polyp Dataset** using this refined strategy provide the most stable diagnostic metrics in the project. Other results from this progressive sequence analysis are summarized in Figures ?? to ?? in the Appendices.

The findings from this final analysis are summarized below:

- **Peak Convergence:** The 16×16 model achieved a stable, peak Accuracy, Precision, Recall, F1-Score and AUC of over 0.9. By preserving the optimizer's momentum, the model was able to reach these levels with significantly less variance compared to the previous incremental approach.
- **Sustained High-Resolution Learning:** The Single-Circuit approach proved vital for the 128×128 resolution. By avoiding the "shocks" of circuit re-initialization, the 15-qubit model maintained a consistent learning trend, ultimately breaking the "random guessing" threshold that hindered the earlier grid search trials.

2.2 ZZ-AQCNN (Analog Quantum Convolutional Neural Networks)

2.2.1 Project Scope and Current Progress

While our overarching experimental design encompasses multi-kernel ensembles (4-kernel and 8-kernel configurations) to maximize topological diversity, the computational intensity of simulating analog Hamiltonian dynamics has necessitated a phased approach. This report focuses on Phase 1: establishing a rigorous **1-Kernel proof-of-concept** for the ZZ-AQCNN architecture. The enhanced multi-kernel configurations are currently computing and represent ongoing future work.

images/oqi_logo.png

images/oqi_text.png

images/Progressive Sequence Single Circuit/16x16_continuousLoss.png

images/footer.jpg

Figure 13: Continuous Training Loss for 16×16 resolution.

images/oqi_logo.png

images/oqi_text.png

images/Progressive Sequence Single Circuit/16x16_minLoss.png

Figure 14: Minimum Training Loss for 16×16 resolution.

images/footer.jpg

images/Progressive Sequence Single Circuit/time.png

Figure 15: Average Training Time per Epoch vs. Image Resolution (Single-Circuit Progressive Sequence).

images/oqi_logo.png

images/oqi_text.png

images/Progressive Sequence Single Circuit/16x16.png

images/footer.jpg

Figure 16: Single-Circuit Progressive Sequence results for 16×16 resolution.

2.2.2 Comparative Performance Matrix

The primary objective of our experiments is to quantify the performance gain provided by the ZZ entanglement correlators and the shift to analog Hamiltonian encoding. We compare our results against a Classical Baseline and the Original DAQCNN [?].

Table 2: ZZ-AQCNN Performance Matrix on BreastMNIST. Results are reported as mean $\pm$ standard deviation across 10 validation seeds for the top-performing trials.

Config / Metric	Classical Baseline	Original DAQCNN (No ZZ)	Digital + ZZ (Enhanced)	Analog + ZZ (ZZ-AQCNN)
1-Kernel (Best)				
Test Acc (%)	0.0 ± 0.0	86.7 ± 1.3	87.9 ± 0.9	86.4 ± 1.0
ROC-AUC	0.00 ± 0.0	0.88 ± 0.01	0.91 ± 0.01	0.90 ± 0.01
4-Kernels				
Test Acc (%)	0.0 ± 0.0	88.1 ± 1.0	0.0 ± 0.0	0.0 ± 0.0
ROC-AUC	0.00 ± 0.0	0.89 ± 0.01	0.00 ± 0.0	0.00 ± 0.0
8-Kernels				
Test Acc (%)	—	—	0.0 ± 0.0	0.0 ± 0.0
ROC-AUC	—	—	0.00 ± 0.0	0.00 ± 0.0

2.2.3 Optimal Hyperparameters (1-Kernel Digital + ZZ)

The hyperparameter search for the 1-Kernel Digital + ZZ configuration yielded a peak accuracy of 87.9% using the **vertical** topology. The optimal classical classification head configuration utilized a batch size of 32, a ReLU activation function, and a dropout rate of approximately 0.566. For optimization, the search identified a learning rate of 0.00128, a weight decay of 0.0001, and a gradient clipping threshold of 7.83. The model converged effectively utilizing an early stopping patience of 71 epochs and a cosine annealing learning rate scheduler over a maximum of 100 epochs. This specific tuning was critical in mitigating overfitting on the challenging grayscale BreastMNIST dataset while fully leveraging the expanded ZZ feature space.

2.2.4 Feature Diversity and Kernel Alignment

To justify the use of multiple kernel topologies, we analyze the structural redundancy of the quantum feature maps using Centered Kernel Alignment (CKA). A lower CKA score between two topologies indicates that they are extracting complementary information from the same image patch.

2.2.5 The Role of Higher-Order Correlators

We investigate whether the classical classification head effectively utilizes the expanded feature space provided by the ZZ measurements.

images/oqi_logo.png

images/oqi_text.png

images/aqcnn/cka_heatmap.png

Figure 17: Placeholder: CKA Similarity Heatmap between the 8 topologies. We expect to see low correlation between directional kernels (e.g., Horizontal vs. Vertical).

2.2.6 Stability and Convergence

We evaluate the robustness of the ZZ-AQCNN framework across 10 independent validation seeds, as detailed in Figure ??.

images/footer.jpg

3 Discussion and Conclusions

3.1 Quantum Neural Networks (QNNs)

The simulation phase of the G-quAI project has provided a comprehensive roadmap for the deployment of Quantum Neural Networks in gastrointestinal diagnostics. By systematically navigating from CPU-based baselines to high-performance GPU-accelerated pipelines, we have identified the fundamental limits and the optimization strategies required for medical-grade Quantum AI.

The most significant learning from Phase 3 is the identification of a complexity threshold in the FRQI-encoded Hilbert space.

- **Impact of Noise and Resolution:** While the QNN proved remarkably resilient to Gaussian noise at lower resolutions (4×4 to 16×16), we observed a distinct performance decay at higher scales. The "Precision-Recall Mismatch" identified in Section 2, where the model maintains high Precision but suffers in Recall, reveals that the current Hardware-Efficient Ansatz (HEA) tends to become "overly conservative"

images/oqi_logo.png

images/oqi_text.png

images/aqcnn/feature_importance.png

images/footer.jpg

Figure 18: Placeholder: Feature Importance Ranking. This plot will quantify the contribution of ZZ entanglement correlators to the final classification decision.

images/oqi_logo.png

images/oqi_text.png

images/aqcnn/violin_seeds.png

images/aqcnn/learning_curves.png

(a) Accuracy Distribution (Seeds 11-20)

(b) Training Convergence

Figure 19: Placeholder: Stability Analysis. (a) Violin plots comparing seed-wise variance across methods. (b) Learning curves comparing the convergence of Classical vs. Quantum-Hybrid models.

as the number of qubits increases. This suggests that while the model learns to identify clear mucosal features, it struggles to navigate the vastly more complex optimization landscape of 13 to 15 qubits.

- **Data Pre-processing and Embeddings:** Our automated CLI pipeline ensured that images were natively generated at the target resolution, avoiding interpolation artifacts. However, the Black and White baseline experiment confirmed that the scaling bottleneck is structural rather than data-driven. While FRQI is efficient for spatial encoding, future work could explore NEQR (Novel Enhanced Quantum Representation) (and other methods of encoding images), to determine if bit-plane intensity encoding offers a more stable gradient path than the probability-amplitude based FRQI.

To overcome the challenges encountered, we developed a hierarchy of optimization techniques that will be vital for the transition to Phase 4:

- **Circuit Construction and Parametric Entanglement:** A major breakthrough was the transition from static CNOT gates to Controlled-RZ (CRZ) gates. By making the entanglement itself a trainable parameter, we allowed the model to "prune" non-essential spatial correlations. This depth-reduction strategy is essential for the NISQ era, where every additional gate introduces decoherence.
- **Gradient Continuity:** The "Single-Circuit" architecture solved the problem of optimizer state loss. By maintaining a constant circuit structure and only "activating" layers, we preserved the Adam optimizer's momentum.

Also, during these phase, several technical hurdles were overcome:

1. **The CPU Simulation Bottleneck:** The initial 60-second per epoch latency on CPU made hyperparameter tuning impossible. We solved this by migrating to a Stateless JAX pipeline, achieving a $120\times$ speedup.
2. **Anomalous Depth Behavior:** We encountered non-monotonic performance across different circuit depths. We concluded that certain depths create "local traps" in the Hilbert space. Our solution for Phase 4 is to utilize the Progressive Summary (Section 2.3.5) to select the "Optimal Depth" rather than blindly increasing complexity.
3. **Optimizer Discontinuities:** The "spikes" in the loss history when adding layers were identified as a loss of gradient history. We solved this by shifting to the Single-Circuit approach, which ensured a seamless training trajectory across 200,000 epochs.

In conclusion, Phase 3 has moved G-quAI from a theoretical concept to a validated diagnostic engine. The achievements presented before demonstrate that Quantum AI can compete with classical benchmarks in specialized medical niches, provided that the optimization and training strategies are tailored to the quantum architecture.

3.2 ZZ-AQCNN (Analog Quantum Convolutional Neural Networks)

The inclusion of ZZ entanglement and the shift to analog Hamiltonian encoding represent a significant step towards more physically realistic and potentially more powerful Quantum CNNs. By including the ZZ correlators, we increase the feature density per kernel from 9 to 45 (for a 3×3 kernel). Preliminary observations suggest that this richer feature space allows the classical classification head to draw more nuanced decision boundaries, particularly in the high-variance regions of the dataset.

A key finding of this work is the trade-off between the precision of digital gate-based encoding and the physical realism of analog Hamiltonian encoding. While digital encoding allows for exact state preparation, the analog mode more closely mirrors the natural evolution of neutral-atom processors. While the dynamic routing approach explored during the transition from QGARS proved challenging, the current results from the Analog ZZ search will define the new state-of-the-art for our architecture on medical images.

4 Impact assessment: Updates to the Full Proposal

4.1 SDG Interlinkage and Synergy

Table ?? shows the interlinkage between the selected SDG targets for the G-quAI use case. Higher values indicate stronger positive influence between targets.

Influencing Targets	3.4	3.8	9.5	10.2	13.3	17.16	Outsum
3.4	0	2	2	2	1	1	8
3.8	1	0	1	1	0	1	4
9.5	2	1	0	1	2	2	8
10.2	2	1	1	0	0	2	6
13.3	1	0	1	1	0	1	4
17.16	3	2	2	2	1	0	10
Insum	9	6	7	7	4	7	

Table 3: SDG interlinkage matrix for the G-quAI use case.

Our analysis using the OQI impact framework highlights that the project's primary focus on SDG 3.4 (Reducing premature mortality from non-communicable diseases) creates a "force multiplier" effect across other goals. As shown in Table ??, the strongest driver of impact is Target 17.16 (Enhance Global Partnership for Sustainable Development), which reflects our collaborative workflow between the University of Coimbra, IT, and our clinical partners.

4.2 Anticipated Impact Flowchart

Figure ?? summarises the expected impact pathway of the G-quAI use case, showing how activities and deliverables lead to short-term and mid-term changes and ultimately to the intended long-term impact.

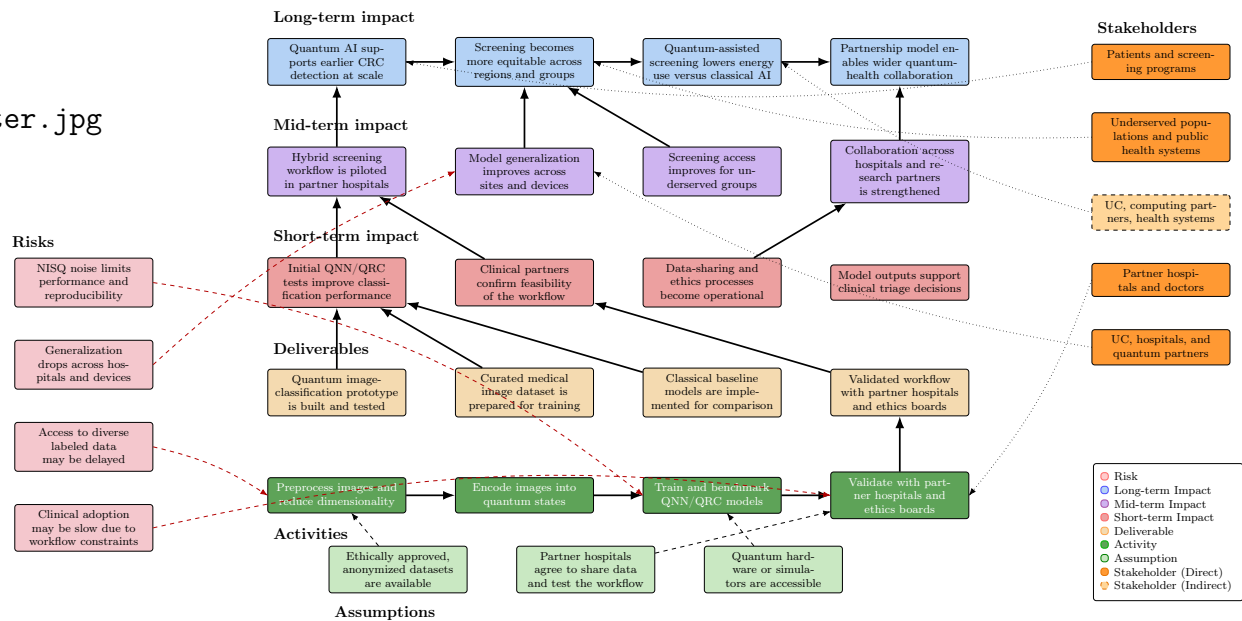


Figure 20: Expected impact pathway for the G-quAI Use Case..

5 Moving to Phase 4

5.1 Feasibility and Resource Estimation for QPU Implementation

The successful completion of Phase 3, which established a robust 1-Kernel proof-of-concept for the ZZ-AQCNN architecture, provides a strong foundation for moving to Phase 4: deployment on real Quantum Processing Units (QPUs). The analog nature of our Hamiltonian encoding and our focus on hardware-native topologies (such as the 2D grid and asymmetric blockade gadgets) make neutral-atom devices like QuEra’s Aquila, accessible via AWS Braket, the ideal target hardware.

To formally request AWS credits and schedule QPU time, we have calculated a precise resource estimation for the inference phase of our model on the BreastMNIST test set.

5.1.1 Patch-Based Workload

The BreastMNIST dataset consists of 28×28 images. Using our 3×3 quantum kernels with a stride of $s = 3$, each image is decomposed into:

$$N_{\text{patches}} = \left(\left\lfloor \frac{28-3}{3} \right\rfloor + 1 \right)^2 = 9 \times 9 = 81 \text{ patches per image.} \quad (5)$$

For a test set of 156 images, a complete inference run for a single kernel requires $156 \times 81 = 12,636$ patch evaluations.

5.1.2 Spatial Multiplexing (Parallel Execution)

A major advantage of our patch-based architecture on neutral-atom arrays is the ability to perform **Spatial Multiplexing**. Since each patch only requires 9 qubits, we do not need to evaluate them one by one sequentially. If we have access to an intermediate-scale processor with 256 qubits (e.g., Aquila), we can instantiate multiple independent kernels simultaneously.

By separating each 9-qubit kernel cluster by a distance significantly larger than the Rydberg blockade radius ($R \gg R_b$), we ensure zero cross-talk between patches. For example, conservatively allocating an effective footprint of 12 qubits of physical space per kernel to ensure total isolation, we can fit:

$$\text{Kernels per task} = \left\lfloor \frac{256}{12} \right\rfloor \approx 21 \text{ parallel patches.} \quad (6)$$

This spatial multiplexing reduces the required quantum tasks from 12,636 to just ≈ 602 tasks.

5.1.3 Shot Count and Precision Estimation

To extract the expected values ($\langle Z \rangle$ and $\langle ZZ \rangle$ correlators) from the quantum state, we must perform multiple projective measurements (shots). The statistical error of the mean (standard error) scales according to Hoeffding’s inequality and the Central Limit Theorem as $\mathcal{O}(1/\sqrt{N_{\text{shots}}})$.

To achieve a desired precision ϵ in our extracted features, the required number of shots is dictated by the formula:

$$N_{\text{shots}} \approx \frac{1}{\epsilon^2} \quad (7)$$

To achieve a robust 5% margin of error ($\epsilon = 0.05$) in our analog feature extraction, we require $N_{\text{shots}} = 1/0.0025 = 400$ shots per task.

5.1.4 Total AWS Resource Request

Combining these factors, the complete resource estimation for running the ZZ-AQCNN inference on the full BreastMNIST test set using 1 kernel is:

- **Total Quantum Tasks:** 602
- **Shots per Task:** 400
- **Total Shots:** $602 \times 400 = 240,800$ shots

This represents a highly feasible and cost-effective computational workload for Phase 4. The ability to spatially multiplex up to 21 patches simultaneously drastically reduces the overhead time (and task-based fees) and makes neutral-atom QPUs perfectly suited for our convolutional approach.

A Extended Results - Quantum Neural Networks (QNNs)

A.1 Extended CPU Simulation Results

This appendix provides the full performance metrics (Accuracy, Precision, Recall, F1, and AUC) for the remaining resolutions tested during the preliminary CPU phase.

images/CPU/04x04.png

Figure 21: Metrics results for 04×04 resolution (CPU Trials, 500 epochs).

images/oqi_logo.png

images/oqi_text.png

images/CPU/08x08.png

Figure 22: Metrics results for 08×08 resolution (CPU Trials, 500 epochs).

images/footer.jpg

images/oqi_logo.png

images/oqi_text.png

images/CPU/32x32.png

Figure 23: Metrics results for 32×32 resolution (CPU Trials, 500 epochs).

images/footer.jpg

A.2 Extended JAX Simulation Results

This section contains the performance metrics for the JAX-accelerated GPU simulations at various resolutions.

images/JAX/08x08.png

Figure 24: Metrics results for 08×08 resolution (JAX Trials, 1,000 epochs).

images/oqi_logo.png

images/oqi_text.png

images/JAX/32x32.png

Figure 25: Metrics results for 32×32 resolution (JAX Trials, 1,000 epochs).

images/footer.jpg

images/oqi_logo.png

images/oqi_text.png

images/JAX/64x64.png

Figure 26: Metrics results for 64×64 resolution (JAX Trials, 1,000 epochs).

images/footer.jpg

images/oqi_logo.png

images/oqi_text.png

A.3 Extended Grid Search Results

This section provides the exhaustive metrics for the Grid Search campaign conducted on the Noisy Synthetic Polyp Dataset across varying resolutions.

images/Grid Search/04x04.png

Figure 27: Grid Search results for 04×04 resolution (1,000 epochs for each combination).

images/footer.jpg

images/oqi_logo.png

images/oqi_text.png

images/Grid Search/08x08.png

Figure 28: Grid Search results for 08×08 resolution (1,000 epochs for each combination).

images/footer.jpg

images/oqi_logo.png

images/oqi_text.png

images/Grid Search/32x32.png

Figure 29: Grid Search results for 32×32 resolution (1,000 epochs for each combination).

images/footer.jpg

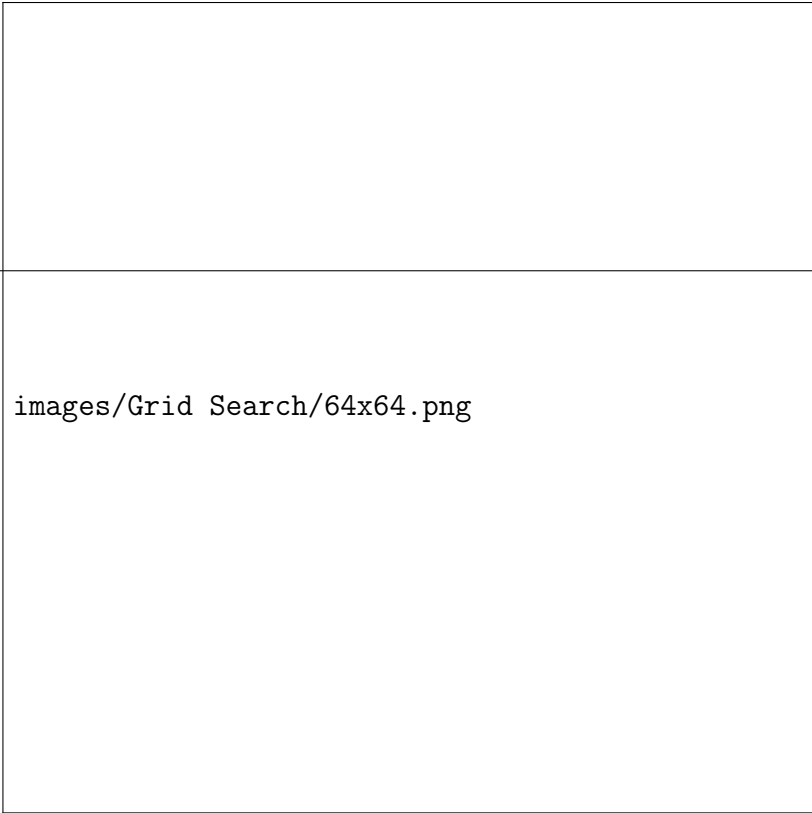
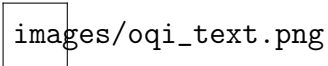
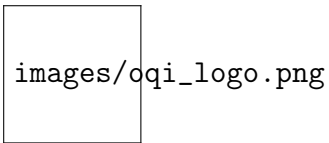
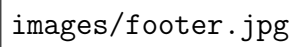


Figure 30: Grid Search results for 64×64 resolution (1,000 epochs for each combination).



images/oqi_logo.png

images/oqi_text.png

images/Grid Search/128x128.png

Figure 31: Grid Search results for 128×128 resolution (1,000 epochs for each combination).

images/footer.jpg

images/oqi_logo.png

images/oqi_text.png

A.4 Extended Fixed-Parameter Grid Search Results

This section provides the performance metrics for the high-duration (5,000 epochs) simulations conducted with a fixed Learning Rate (0.05) and Batch Size (32).

images/Grid Search Fixed/04x04.png

Figure 32: Fixed-parameter Grid Search results for 04×04 resolution (5,000 epochs for each depth)

images/oqi_logo.png

images/oqi_text.png

images/Grid Search Fixed/08x08.png

Figure 33: Fixed-parameter Grid Search results for 08×08 resolution (5,000 epochs for each depth).

images/footer.jpg

images/oqi_logo.png

images/oqi_text.png

images/Grid Search Fixed/32x32.png

Figure 34: Fixed-parameter Grid Search results for 32×32 resolution (5,000 epochs for each depth).

images/footer.jpg

images/oqi_logo.png

images/oqi_text.png

images/Grid Search Fixed/64x64.png

Figure 35: Fixed-parameter Grid Search results for 64×64 resolution (5,000 epochs for each depth).

images/footer.jpg

images/oqi_logo.png

images/oqi_text.png

images/Grid Search Fixed/128x128.png

Figure 36: Fixed-parameter results for 128×128 resolution (5,000 epochs for each depth).

images/footer.jpg

images/oqi_logo.png

images/oqi_text.png

A.5 Extended Black and White Baseline Results

This section provides the metrics for the control experiments conducted on the Simple Black and White Dataset to verify mathematical convergence limits.

images/Grid Search Fixed BW/04x04.png

Figure 37: Black and White Baseline results for 04×04 resolution (3,000 epochs for each depth).

images/Grid Search Fixed BW/08x08.png

Figure 38: Black and White Baseline results for 08×08 resolution (3,000 epochs for each depth).

images/oqi_logo.png

images/oqi_text.png

images/Grid Search Fixed BW/32x32.png

Figure 39: Black and White Baseline results for 32×32 resolution (3,000 epochs for each depth).

images/Grid Search Fixed BW/64x64.png

Figure 40: Black and White Baseline results for 64×64 resolution (3,000 epochs for each depth).

images/oqi_logo.png

images/oqi_text.png

images/Grid Search Fixed BW/128x128.png

Figure 41: Black and White Baseline results for 128×128 resolution (3,000 epochs for each depth).

images/footer.jpg

A.6 Extended Progressive Sequence Results

This section provides the metrics for the Progressive Training campaign (10,000 epochs per layer) using the Warm Start strategy.

images/Progressive Sequence/32x32.png

Figure 42: Progressive Sequence results for 32×32 resolution (10,000 epochs for each depth).

images/oqi_logo.png

images/oqi_text.png

images/Progressive Sequence/32x32_continuousLoss.png

images/footer.jpg

Figure 43: Continuous Training Loss for 32×32 resolution.

images/oqi_logo.png

images/oqi_text.png

images/Progressive Sequence/64x64.png

Figure 44: Progressive Sequence results for 64×64 resolution (10,000 epochs for each depth).

images/footer.jpg

images/oqi_logo.png

images/oqi_text.png

images/Progressive Sequence/64x64_continuousLoss.png

images/footer.jpg

Figure 45: Continuous Training Loss for 64×64 resolution.

images/oqi_logo.png

images/oqi_text.png

images/Progressive Sequence/128x128.png

Figure 46: Progressive Sequence results for 128×128 resolution (10,000 epochs for each depth).

images/footer.jpg

images/oqi_logo.png

images/oqi_text.png

images/Progressive Sequence/128x128_continuousLoss.png

images/footer.jpg

Figure 47: Continuous Training Loss for 128×128 resolution.

A.7 Extended Single-Circuit Progressive Results

This section provides the final performance metrics and continuous loss curves for the Single-Circuit Progressive strategy (10,000 epochs per layer).

images/Progressive Sequence Single Circuit/32x32.png

Figure 48: Single-Circuit Progressive Sequence results for 32×32 resolution (10,000 epochs for each depth).

images/oqi_logo.png

images/oqi_text.png

images/Progressive Sequence Single Circuit/32x32_continuousLoss.png

images/footer.jpg

Figure 49: Continuous Training Loss for 32×32 resolution.

images/oqi_logo.png

images/oqi_text.png

images/Progressive Sequence Single Circuit/64x64.png

Figure 50: Single-Circuit Progressive Sequence results for 64×64 resolution (10,000 epochs for each depth).

images/footer.jpg

images/oqi_logo.png

images/oqi_text.png

images/Progressive Sequence Single Circuit/64x64_continuousLoss.png

images/footer.jpg

Figure 51: Continuous Training Loss for 64×64 resolution.

images/oqi_logo.png

images/oqi_text.png

images/Progressive Sequence Single Circuit/128x128.png

Figure 52: Single-Circuit Progressive Sequence results for 128×128 resolution (10,000 epochs for each depth).

images/footer.jpg

images/oqi_logo.png

images/oqi_text.png

images/Progressive Sequence Single Circuit/128x128_continuousLoss.png

images/footer.jpg

Figure 53: Continuous Training Loss for 128×128 resolution.